

**LAPORAN TUGAS**  
**ANALISIS & IMPLEMENTASI DECISION TREE & NAIVE BAYES**



**Disusun Oleh :**

Rofid Fadhil                      (G1A024065)

**Kelompok :**

1. Abid Abdusalam              (G1A024073)
2. Achmad Rifky Akbar      (G1A024091)

**Mata Kuliah:**

Machine Learning

**Dosen Pengampu:**

Ir. Arie Vatesia, S.T., M.T.I., Ph.D IPP.

**UNIVERSITAS BENGKULU**  
**FAKULTAS TEKNIK**  
**PROGRAM STUDI INFORMATIKA**  
**2026**

## SOAL DAN PEMBAHASAN

### BAGIAN A: ANALISIS KONSEP - Individu

1. Jelaskan kapan Decision Tree lebih cocok digunakan dibanding Naive Bayes untuk kasus prediksi kelulusan ini. Beri 2 alasan.

Jawab:

Decision Tree lebih cocok untuk kasus prediksi kelulusan karena kemampuannya menangani interaksi non-linear dan kondisional antar fitur tanpa memerlukan asumsi distribusi data. Dalam realitas akademik, kelulusan sering kali bergantung pada kombinasi syarat bertingkat. Misalnya, mahasiswa dengan Kehadiran rendah mungkin tetap bisa lulus tepat waktu jika IPK-nya sangat tinggi dan Jam\_Belajar mandirinya optimal. Decision Tree secara alami menangkap hierarki logika seperti ini melalui percabangan bersyarat, sementara Naive Bayes akan kesulitan karena memproses probabilitas setiap fitur secara terpisah.

Alasan kedua adalah tingkat interpretabilitas model yang sangat krusial bagi manajemen kampus. Decision Tree menghasilkan aturan keputusan eksplisit (aturan *if-then*) yang mudah divisualisasikan dan dipahami oleh pihak non-teknis, seperti Dosen Pembimbing Akademik. Dengan aturan visual seperti "Jika Kehadiran < 75% dan IPK < 3.0 maka Tidak Lulus", pihak kampus dapat dengan mudah merancang kebijakan atau memberikan teguran secara transparan. Sebaliknya, Naive Bayes hanya menghasilkan nilai probabilitas akhir yang cenderung sulit diterjemahkan ke dalam tindakan intervensi langsung di lapangan.

Decision Tree juga lebih cocok ketika terdapat hubungan antarfitur yang tidak sederhana, sedangkan Naive Bayes mengasumsikan bahwa setiap fitur bekerja secara independen.

2. Asumsi "independen" di Naive Bayes. Sebutkan 2 pasang fitur di dataset yang kemungkinan melanggar asumsi ini dan jelaskan kenapa.

Jawab:

Algoritma Naive Bayes mengasumsikan bahwa setiap fitur berdiri sendiri dan tidak memiliki korelasi satu sama lain (independensi kondisional). Pada dataset ini, asumsi tersebut kemungkinan besar dilanggar oleh pasangan fitur **IPK dan Jam\_Belajar**. Secara logis, durasi jam belajar seorang mahasiswa memiliki pengaruh kausalitas yang kuat terhadap tinggi rendahnya IPK yang diperoleh. Jika Naive Bayes memproses kedua fitur ini, model akan memperlakukannya sebagai dua informasi yang terpisah, sehingga terjadi penghitungan ganda (*double counting*) pada faktor usaha akademik yang akan mendistorsi probabilitas akhirnya.

Pasangan kedua yang rentan melanggar asumsi adalah **Penghasilan\_Ortu dan Status\_Beasiswa**. Dalam sistem administrasi perguruan tinggi, mayoritas program beasiswa menjadikan batas maksimal penghasilan orang tua sebagai syarat utama kelayakan penerima. Artinya, jika mahasiswa berstatus menerima beasiswa, bisa dipastikan penghasilan orang tuanya berada pada rentang menengah ke bawah. Memasukkan kedua fitur ini secara bersamaan akan membuat Naive Bayes menjadi terlalu percaya diri (*overconfident*), karena model mengalikan probabilitas dari dua fitur yang pada dasarnya merepresentasikan satu kondisi status sosial-ekonomi yang sama.

3. Apa risiko overfitting pada Decision Tree di dataset ini? Bagaimana cara mengatasinya?

Jawab:

Risiko *overfitting* pada Decision Tree sangat tinggi untuk dataset ini karena jumlah datanya yang relatif kecil (hanya 500 baris) namun memiliki 7 fitur yang bervariasi. Jika dibiarkan tumbuh tanpa batasan, algoritma akan terus memecah cabang hingga setiap data

latih terklasifikasi dengan murni. Akibatnya, model tidak lagi mempelajari pola umum kelulusan, melainkan "menghafal" *noise* dan anomali unik dari satu atau dua mahasiswa tertentu. Model yang *overfit* ini mungkin meraih akurasi sempurna di data latih, tetapi performanya akan anjlok saat digunakan untuk memprediksi mahasiswa angkatan baru karena aturannya kelewat spesifik.

Untuk mengatasi risiko ini, kita perlu menerapkan teknik pemangkasan (*pruning*), khususnya dengan membatasi *hyperparameter* saat membangun model. Langkah paling umum adalah membatasi kedalaman maksimum pohon menggunakan parameter `max_depth` (misalnya maksimal 3 tingkat). Selain itu, kita bisa menggunakan `min_samples_split` untuk menetapkan batas minimum jumlah mahasiswa dalam satu simpul (*node*) sebelum simpul tersebut diizinkan untuk dipecah menjadi cabang baru. Batasan ini memaksa model untuk mengabaikan pola-pola minoritas, sehingga model yang dihasilkan lebih sederhana, general, dan stabil ketika dihadapkan pada data baru.

## BAGIAN B: IMPLEMENTASI & EVALUASI - Kelompok 2-3 orang

1. Preprocessing: Handle missing value, encoding, train-test split 80:20

Jawab:

```
print(df.isnull().sum())

*** IPK          0
    Kehadiran    0
    Jam_Belajar   0
    Organisasi    0
    Penghasilan_Ortu 0
    Jenis_Kelamin 0
    Status_Beasiswa 0
    Status_Kelulusan 0
    dtype: int64
```

```
import pandas as pd

df = pd.read_excel("Mahasiswa_Lulus.xlsx")

display(df.head())
```

```
X = pd.get_dummies(
    X,
    columns=[
        "Organisasi",
        "Jenis_Kelamin",
        "Status_Beasiswa"
    ]
)

y = y.map({
    "LULUS": 1,
    "TIDAK LULUS": 0
})

display(X.head())
display(y.head())
```

```
X = df.drop("Status_Kelulusan", axis=1)
y = df["Status_Kelulusan"]
```

|   | IPK  | Kehadiran | Jam_Belajar | Penghasilan_Ortu | Organisasi_No | Organisasi_Yes | Jenis_Kelamin_Female | Jenis_Kelamin_Male | Status_Beasiswa_No | Status_Beasiswa_Yes |
|---|------|-----------|-------------|------------------|---------------|----------------|----------------------|--------------------|--------------------|---------------------|
| 0 | 3.93 | 100       | 5           | 15000            | True          | False          | False                | True               | False              | True                |
| 1 | 3.90 | 90        | 2           | 20000            | True          | False          | False                | True               | False              | True                |
| 2 | 3.59 | 98        | 3           | 100000           | False         | True           | True                 | False              | False              | True                |
| 3 | 3.78 | 100       | 5           | 40000            | False         | True           | True                 | False              | True               | False               |
| 4 | 3.81 | 100       | 4           | 50000            | True          | False          | False                | True               | False              | True                |

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Data training:", X_train.shape)
print("Data testing:", X_test.shape)

*** Data training: (400, 10)
    Data testing: (100, 10)
```

Penjelasan:

Tahap prapemrosesan data menentukan kualitas representasi data sebelum diumpungkan ke model *machine learning*. Berdasarkan pengecekan awal, tidak terdapat nilai kosong pada dataset sehingga tidak perlu dilakukan imputasi data. Fitur kategorikal (Organisasi, Jenis\_Kelamin, dan Status\_Beasiswa) diubah ke format angka menggunakan metode *One-Hot Encoding* (pd.get\_dummies) agar tidak menciptakan asumsi hierarki palsu, sedangkan variabel target (Status\_Kelulusan) dipetakan secara biner menjadi 1 dan 0. Sementara itu, pembagian data latih dan uji dilakukan dengan proporsi 80:20 (400 data latih, 100 data uji) menggunakan parameter stratify=y guna memastikan rasio kelas target tetap seimbang di kedua subset.

## 2. Modeling:

Bangun model 'DecisionTreeClassifier' dengan 'max\_depth=3' dan 'max\_depth=None'

Bangun model 'GaussianNB'

Jawab:

```
from sklearn.tree import DecisionTreeClassifier

model_dt_3 = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)

model_dt_3.fit(X_train, y_train)

y_pred_dt_3 = model_dt_3.predict(X_test)

model_dt_none = DecisionTreeClassifier(
    max_depth=None,
    random_state=42
)

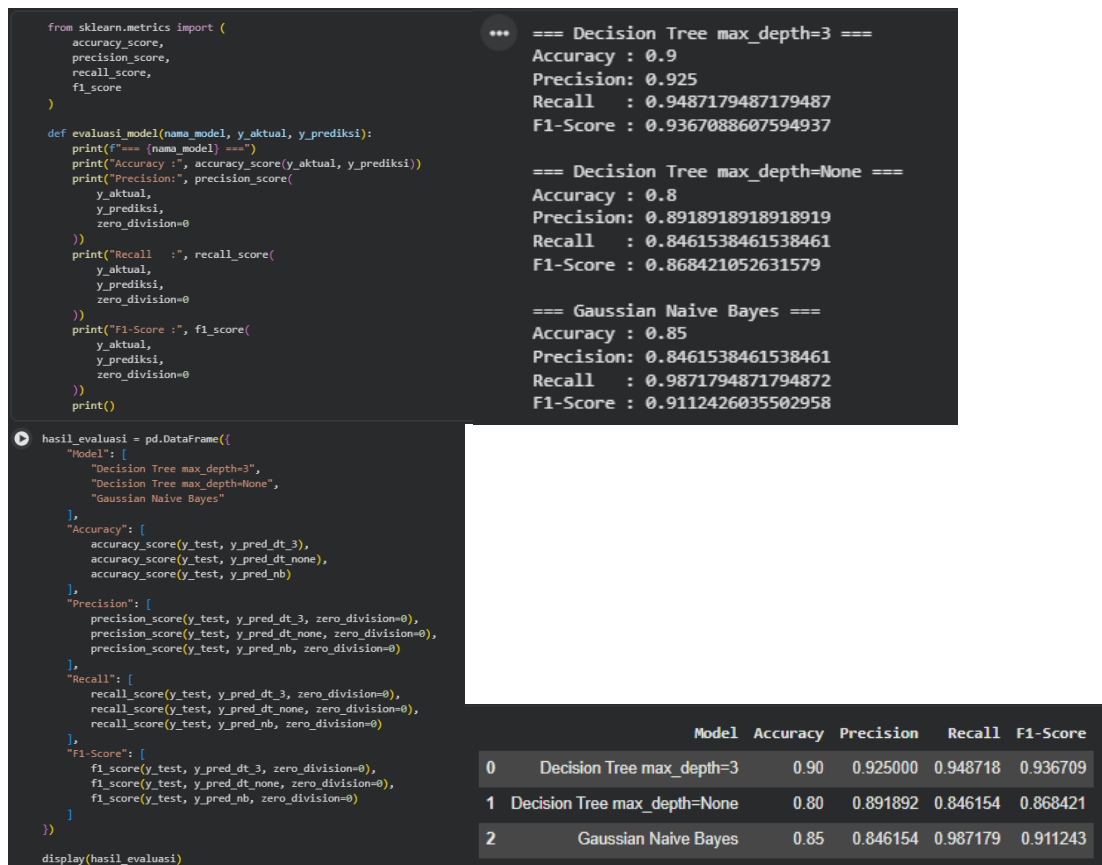
model_dt_none.fit(X_train, y_train)

y_pred_dt_none = model_dt_none.predict(X_test)
```

### Penjelasan :

Tahap pemodelan membandingkan tiga pendekatan berbeda. Model *Decision Tree* dengan parameter max\_depth=3 dibangun untuk membuat aturan keputusan yang tergeneralisasi dan mencegah terjadinya *overfitting* pada data latih. Sebaliknya, model *Decision Tree* dengan max\_depth=None dibangun untuk melihat efek saat pohon dibiarkan tumbuh maksimal mempelajari detail terkecil pada data. Sebagai pembanding algoritma probabilitas, *Gaussian Naive Bayes* digunakan karena sangat efisien dalam melakukan klasifikasi dengan asumsi distribusi normal pada fitur numerik.

## 3. Evaluasi: Hitung Akurasi, Precision, Recall, F1-Score untuk ke-3 model. Buat Confusion Matrix



### Penjelasan:

Berdasarkan hasil metrik, *Decision Tree* dengan `max_depth=3` merupakan model terbaik yang menghasilkan tingkat Akurasi 90% dan F1-Score 93.67%. Model *Decision Tree* tanpa batasan (`max_depth=None`) terbukti mengalami *overfitting* karena akurasinya turun signifikan menjadi 80% pada data uji. *Gaussian Naive Bayes* memberikan performa cukup baik (Akurasi 85%) dengan tingkat *Recall* tertinggi (98.71%), yang berarti model ini sangat sensitif dalam mendeteksi target LULUS. *Confusion Matrix* yang dibuat memvalidasi sebaran prediksi tersebut dengan memetakan jumlah tebakan benar dan salah secara spesifik.

### 4. Visualisasi:

Plot Pohon Keputusan dengan `'max_depth=3'`

Plot perbandingan metrik 3 model dalam bentuk bar chart

Jawab:

```

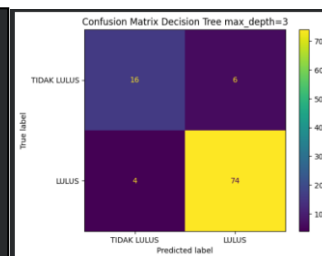
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

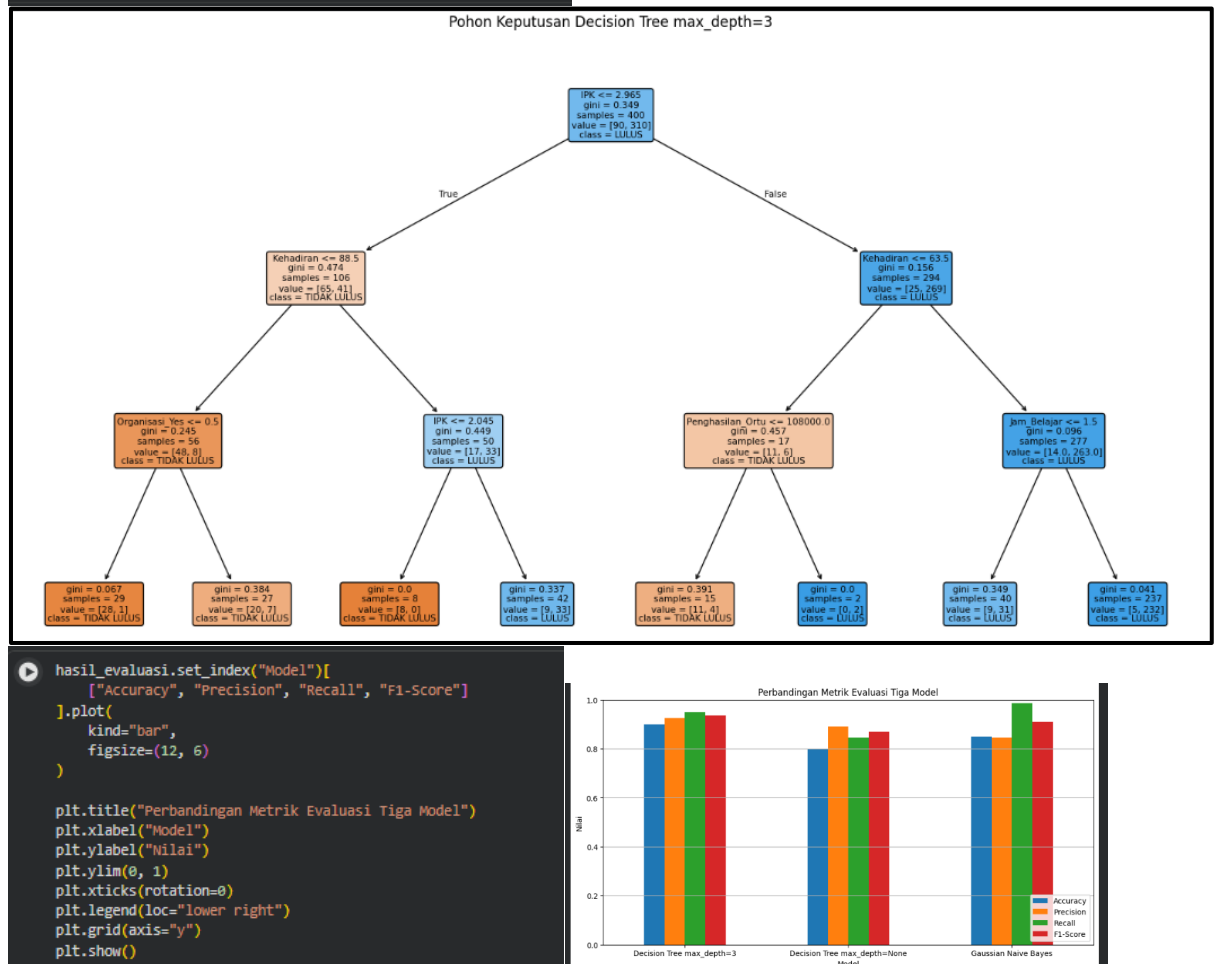
cm_dt_3 = confusion_matrix(y_test, y_pred_dt_3)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm_dt_3,
    display_labels=["TIDAK LULUS", "LULUS"]
)

disp.plot()
plt.title("Confusion Matrix Decision Tree max_depth=3")
plt.show()

```





## Penjelasan:

Plot Pohon Keputusan (max\_depth=3) divisualisasikan untuk memberikan transparansi terkait bagaimana model mengambil keputusan, memperlihatkan fitur apa saja (seperti batasan IPK atau Kehadiran) yang menjadi akar pemisah utama. Selain itu, grafik batang (*bar chart*) digunakan untuk membandingkan keempat metrik (Akurasi,

Presisi, Recall, dan F1-Score) secara berdampingan, sehingga keunggulan *Decision Tree* (max\_depth=3) atas model lainnya terlihat jelas dan mudah diinterpretasikan.

5. **Analisis Error:** Ambil 5 data yang salah prediksi. Jelaskan kenapa menurutmu model salah

Jawab:

```

hasil_prediksi = X_test.copy()

hasil_prediksi["Aktual"] = y_test
hasil_prediksi["Prediksi"] = y_pred_dt_3

data_salah_lengkap = data_salah.copy()

data_salah_lengkap["Aktual"] = data_salah_lengkap["Aktual"].map({
    1: "LULUS",
    0: "TIDAK LULUS"
})

data_salah_lengkap["Prediksi"] = data_salah_lengkap["Prediksi"].map({
    1: "LULUS",
    0: "TIDAK LULUS"
})

print(
    "Jumlah data yang salah diprediksi:",
    len(data_salah)
)

display(data_salah_lengkap.head(5))

```

| IPK | Kehadiran | Jam_Belajar | Penghasilan_Ortu | Organisasi_No | Organisasi_Yes | Jenis_Kelamin_Female | Jenis_Kelamin_Male | Status_Beasiswa_No | Status_Beasiswa_Yes | Aktual | Prediksi |
|-----|-----------|-------------|------------------|---------------|----------------|----------------------|--------------------|--------------------|---------------------|--------|----------|
| 19  | 3.15      | 90          | 2                | 40000         | True           | False                | True               | False              | True                | 0      | 1        |
| 263 | 2.94      | 100         | 1                | 121000        | False          | True                 | False              | True               | False               | 0      | 1        |
| 475 | 2.80      | 90          | 2                | 30000         | True           | False                | True               | False              | True                | 0      | 1        |
| 110 | 2.77      | 95          | 1                | 40000         | False          | True                 | False              | True               | False               | 0      | 1        |
| 160 | 3.15      | 82          | 2                | 40000         | True           | False                | True               | False              | True                | 0      | 1        |

```

Jumlah data yang salah diprediksi: 10

```

| IPK | Kehadiran | Jam_Belajar | Penghasilan_Ortu | Organisasi_No | Organisasi_Yes | Jenis_Kelamin_Female | Jenis_Kelamin_Male | Status_Beasiswa_No | Status_Beasiswa_Yes | Aktual      | Prediksi |
|-----|-----------|-------------|------------------|---------------|----------------|----------------------|--------------------|--------------------|---------------------|-------------|----------|
| 19  | 3.15      | 90          | 2                | 40000         | True           | False                | True               | False              | True                | TIDAK LULUS | LULUS    |
| 263 | 2.94      | 100         | 1                | 121000        | False          | True                 | False              | True               | False               | TIDAK LULUS | LULUS    |
| 475 | 2.80      | 90          | 2                | 30000         | True           | False                | True               | False              | True                | TIDAK LULUS | LULUS    |
| 110 | 2.77      | 95          | 1                | 40000         | False          | True                 | False              | True               | False               | TIDAK LULUS | LULUS    |
| 160 | 3.15      | 82          | 2                | 40000         | True           | False                | True               | False              | True                | TIDAK LULUS | LULUS    |

### Penjelasan:

Analisis pada data yang salah diprediksi oleh *Decision Tree* (max\_depth=3) mengungkap adanya 10 data *error* yang seluruhnya berjenis *False Positives* (model memprediksi "LULUS", namun aktualnya "TIDAK LULUS"). Mengambil 5 sampel teratas, kesalahan ini terjadi karena profil mahasiswa tersebut memiliki angka yang sekilas masuk dalam kriteria lulus (seperti IPK 2.77 - 3.15 dan Kehadiran 82 - 100). Model mengalami *bias* terhadap angka-angka dominan kelas mayoritas ini dan gagal menangkap faktor-faktor minoritas yang menjadi penyebab asli mahasiswa tersebut tidak lulus.

### BAGIAN C: REKOMENDASI & ETIKA - Kelompok

1. Rekomendasi Model: Dari 3 model, mana yang kamu pilih untuk dipakai kampus? Jelaskan dengan data dari Bagian B

Jawab:

Dari tiga model yang diuji, saya memilih *Decision Tree* dengan max\_depth=3 untuk digunakan oleh pihak kampus. Berdasarkan hasil evaluasi pada data testing, model ini memperoleh accuracy 90%, precision 92,5%, recall 94,87%, dan F1-Score 93,67%. Nilai tersebut lebih baik dibandingkan *Decision Tree* dengan max\_depth=None yang memperoleh accuracy 80% dan F1-Score 86,84%, serta *Gaussian Naive Bayes* yang memperoleh accuracy 85% dan F1-Score 91,12%.

Selain memiliki hasil evaluasi paling baik, *Decision Tree* max\_depth=3 juga lebih mudah dijelaskan kepada pihak kampus karena hasilnya berbentuk aturan percabangan. Contohnya, model dapat menentukan kelulusan berdasarkan kombinasi IPK, kehadiran, dan jam belajar. Model ini juga lebih aman dibandingkan *Decision Tree* max\_depth=None karena pohonnya dibatasi sehingga risiko overfitting lebih kecil. Dari confusion matrix,

model `max_depth=3` salah memprediksi 10 data, yaitu 6 mahasiswa yang sebenarnya tidak lulus tetapi diprediksi lulus dan 4 mahasiswa yang sebenarnya lulus tetapi diprediksi tidak.

2. Bias & Etika: Fitur "Penghasilan\_Ortu" dan "Jenis\_Kelamin" rawan bias. Apa dampaknya jika model dipakai untuk beasiswa? Bagaimana mitigasinya?

Jawab:

Fitur Penghasilan\_Ortu dan Jenis\_Kelamin berpotensi menimbulkan bias apabila model digunakan untuk menentukan penerima beasiswa. Misalnya, mahasiswa dari keluarga berpenghasilan rendah dapat lebih sering diprediksi tidak lulus karena kondisi ekonominya, padahal mahasiswa tersebut sebenarnya memiliki prestasi akademik yang baik. Sebaliknya, mahasiswa dari keluarga berpenghasilan tinggi bisa saja memperoleh prediksi yang lebih menguntungkan..

Dampaknya, mahasiswa bisa kehilangan kesempatan memperoleh beasiswa bukan karena prestasi atau kebutuhan yang sebenarnya, tetapi karena karakteristik pribadi dan kondisi keluarganya. Mitigasinya adalah menghapus atau membatasi penggunaan fitur yang berpotensi bias, terutama Jenis\_Kelamin, serta menjadikan Penghasilan\_Ortu hanya sebagai salah satu indikator kebutuhan ekonomi, bukan penentu tunggal. Kampus juga perlu melakukan pengujian fairness dengan membandingkan hasil prediksi berdasarkan kelompok jenis kelamin dan kelompok penghasilan.

3. Simulasi: Jika 1 mahasiswa punya IPK tinggi tapi Kehadiran rendah, prediksi kedua model apa? Kenapa bisa beda?

Jawab:

Untuk simulasi, digunakan mahasiswa dengan karakteristik:

- IPK: 3,8
- Kehadiran: 50%
- Jam belajar: 4 jam
- Penghasilan orang tua: Rp5.000.000
- Tidak mengikuti organisasi
- Tidak menerima beasiswa

Berdasarkan simulasi menggunakan model yang sama seperti pada Bagian B, Decision Tree `max_depth=3`, Decision Tree `max_depth=None`, dan Gaussian Naive Bayes sama-sama memprediksi mahasiswa tersebut sebagai LULUS.

Hasil tersebut bisa terjadi karena IPK yang tinggi memberikan pengaruh kuat terhadap prediksi kelulusan. Meskipun kehadirannya rendah, nilai IPK 3,8 membuat karakteristik mahasiswa tersebut lebih dekat dengan kelompok mahasiswa yang lulus. Decision Tree mengambil keputusan berdasarkan aturan percabangan yang terbentuk dari data training, sedangkan Naive Bayes menghitung peluang kelulusan berdasarkan kontribusi masing-masing fitur. Prediksi kedua metode dapat berbeda apabila IPK tinggi tidak cukup untuk mengimbangi kehadiran rendah menurut aturan Decision Tree atau perhitungan probabilitas Naive Bayes. Namun, pada simulasi ini, ketiga model memberikan hasil yang sama, yaitu LULUS.

|                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------|
| Link akses Github: <a href="https://github.com/r0tky/Tugas1_MachineLearning">https://github.com/r0tky/Tugas1_MachineLearning</a> |
|----------------------------------------------------------------------------------------------------------------------------------|